

IUT / IFRI — Université de Parakou

Master Génie Logiciel — Théorie des langages & calculabilité

De l'automate fini à la machine universelle

Projet intégrateur — bibliothèque `formlang`

Énoncé étudiant — cinq journées, une seule bibliothèque, quatre applications.

- **Durée** : 1 semaine (5 jours + soutenance)
- **Binôme**
- **Langage** : Python 3.10+ (`pytest` seul).
- **Rendu** : dépôt Git `NOM1_NOM2_formlang/` + rapport PDF + soutenance.

Auteur du sujet : **Dr. Mêton ATINDEHOU**

Charte IUT/IFRI — bleu #1A3A6B / orange #E07B20

Table des matières

1	Présentation générale	4
1.1	Le fil rouge	4
1.2	La hiérarchie de Chomsky, étage par étage	4
1.3	Une bibliothèque, quatre applications	4
1.4	Pré-requis	5
1.5	Organisation de la semaine & barème global	5
1.6	Méthode de travail (à lire avant de commencer)	5
1.7	Livrables	6
2	Jour 1 — Étage régulier : automates finis & transducteurs	7
2.1	Contexte	7
2.2	Notions nouvelles	7
2.2.1	AFD et AFN	8
2.2.2	Déterminisation par construction des sous-ensembles	8
2.2.3	Minimisation (Moore)	9
2.2.4	Transducteur fini (FST) : normaliser le <i>leet</i>	9
2.2.5	Une limite déjà visible : one-way vs two-way	9
2.3	A. Questions théoriques (à rédiger) — 2 pts	9
2.4	B. Exercices de programmation — 2 pts	10
3	Jour 2 — Étage hors-contexte : grammaires & automate à pile	11
3.1	Contexte	11
3.2	Notions nouvelles	11
3.2.1	Le lemme de l'étoile (pompage)	11
3.2.2	Grammaire hors-contexte et arbre de dérivation	12
3.2.3	Ambiguïté	12
3.2.4	La pile : mémoire non bornée mais disciplinée	12
3.3	A. Questions théoriques (à rédiger) — 2,5 pts	13
3.4	B. Exercices de programmation — 1,5 pt	13
4	Jour 3 — Automates d'arbres (pivot) & hash-consing	14
4.1	Contexte	14
4.2	Notions nouvelles	14
4.2.1	Alphabet classé, termes, et automate d'arbres ascendant (BUTA)	15
4.2.2	Exécution ascendante — exemple morphologique	15
4.2.3	Pourquoi un AFD <i>de mots</i> échoue : la frontière ne suffit pas	15
4.2.4	Le même moteur pour le Shield (100 % structurel)	16
4.2.5	Clôture par intersection (produit) et hash-consing (DAG)	16
4.3	A. Questions théoriques (à rédiger) — 2 pts	17
4.4	B. Exercices de programmation — 3 pts	17
5	Jour 4 — Calculabilité : machine de Turing & machine universelle	19

5.1	Contexte	19
5.2	Notions nouvelles	19
5.2.1	La machine de Turing	20
5.2.2	Une opération <i>est</i> une machine : trace de ADD sur 11+1	20
5.2.3	Lire une table par <i>phases</i> : la soustraction	20
5.2.4	Codage $\langle M \rangle$ et machine universelle	21
5.3	A. Questions théoriques (à rédiger) — 2 pts	21
5.4	B. Exercices de programmation — 2 pts	21
6	Jour 5 — Intégration & Myhill–Nerode	23
6.1	Contexte	23
6.2	Notions nouvelles	23
6.2.1	Intégration : deux chaînes, un même cœur	23
6.2.2	La congruence de Myhill–Nerode	23
6.2.3	En pratique : signatures sur suffixes témoins	24
6.3	A. Questions théoriques (à rédiger) — 1 pt	24
6.4	B. Exercices de programmation — 2 pts	24
7	Bibliographie (vérifiée)	26

1 Présentation générale

1.1 Le fil rouge

Ce projet pose **une seule question**, à cinq niveaux de difficulté croissante : *reconnaître une structure* — depuis un simple motif dans un mot jusqu'à n'importe quel calcul. À chaque étage, une leçon se répète :

*plus la structure à reconnaître est riche, plus il faut de **mémoire** — et il existe un seuil (la machine universelle) où l'on gagne l'universalité mais où l'on **perd la décidabilité**.*

On ne construit pas « trois TP séparés » mais **un cœur unique** (`formlang/`) que l'on **réutilise** : deux applicationsinstancient le même automate d'arbres, une troisième partage la structure des mêmes termes, une quatrième décrit des tables de machines de Turing exécutées par une machine universelle. Cette *réutilisation* est le cœur de l'évaluation.

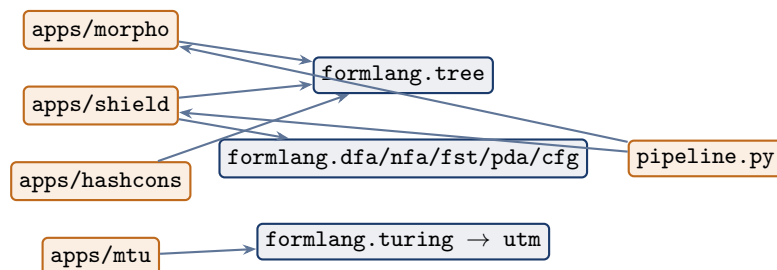
1.2 La hiérarchie de Chomsky, étage par étage

Jour	Étage	Pouvoir de reconnaissance	Mémoire
1	Régulier	langages réguliers	état fini
2	Hors-contexte	langages algébriques	pile (LIFO)
3	Arbres	langages d'arbres réguliers	un état par nœud
4	Calculable	rékursivement énumérable	ruban libre
5	Transversal	—	(congruence, intégration)

Un mot $a_1 \cdots a_n$ est un arbre dégénéré (branche unaire) ; un automate fini est un automate d'arbres « tout en arité 1 ». Vous le **vérifierez en code**.

1.3 Une bibliothèque, quatre applications

Les applications n'ont pas le droit de réécrire un automate : elles **instancient** ceux de `formlang`. C'est la règle « gate » la plus importante du projet.



`apps/morpho`
morphologie — automate d'arbres (BUTA).

apps/shield

AttackDecomposer — automate d'arbres, 100 % structurel.

apps/hashcons

partage de structure (DAG) sur `formlang.tree.Term`.

apps/mtu

machine universelle U + calculatrice unaire *exécutée par* U .

Cadre éthique

Le volet Shield porte **exclusivement** sur la **structure formelle** d'un système de *défense*. Tous les « prompts » sont des suites de **jetons abstraits** ($a, o, r, e, [,], (,)$). **Aucune attaque réelle, aucun contenu sensible**. Tout rendu contenant un exemple d'attaque réelle est hors-sujet et pénalisé.

Gate (règle non négociable)

- *Anti-copier-coller* : une app qui **redéfinit** un automate au lieu de l'instancier perd les points d'intégration de la journée.
- *Honnêteté intellectuelle* : toute borne de complexité doit être **démontrée ou référencée** (jamais « de mémoire »). Reportez vos **vrais** chiffres de sortie.

1.4 Pré-requis

Langages formels ; expressions régulières ; grammaires ; automates finis (AFD/AFN) ; détermination ; minimisation ; notions de calculabilité. Le projet **introduit** les automates d'arbres (comme généralisation des automates finis) et la machine universelle (comme aboutissement).

1.5 Organisation de la semaine & barème global

Jour	Thème	Tests cibles	Pts
1	Régulier & transducteurs	<code>test_regular</code>	4
2	Hors-contexte (pompage, CFG, PDA)	<code>test_cfg_nerode</code> , <code>test_regular</code>	4
3	Arbres + hash-consing	<code>test_tree</code> , <code>test_hashcons</code>	5
4	Calculabilité (MT, MTU, calculatrice)	<code>test_turing</code> , <code>test_calc</code>	4
5	Intégration & Myhill–Nerode	<code>test_pipeline</code> , <code>test_cfg_nerode</code>	3
Total			20
Bonus (Thompson, CYK, minimisation BUTA, infixation, binaire)			+6

1.6 Méthode de travail (à lire avant de commencer)

- Lancez `pytest -q` **immédiatement** : vous verrez N échecs (`NotImplementedError`). C'est normal.

- Chaque # TODO (Ex.) du code renvoie à une question **E...** d'une fiche-jour. Un exercice est **validé** quand son test passe au vert.
- Travaillez *étage par étage* : ne passez au jour suivant que lorsque les tests du jour courant sont verts.
- Le rapport se rédige *en même temps* que le code (traces, preuves, mesures), pas à la fin.

1.7 Livrables

1. Dépôt Git NOM1_NOM2_formlang/ : formlang/, apps/, pipeline.py, tests/ complétés et **qui passent**; joindre `pytest -q > out_tests.txt` et `python pipeline.py > out_demo.txt`.
2. Rapport PDF (gabarit RAPPORT_TEMPLATE.md).
3. Soutenance (10 min) : démonstration bout-en-bout + un cas par étage.

2 Jour 1 — Étage régulier : automates finis & transducteurs

Modules : formlang/dfa.py, nfa.py, fst.py — Apps : apps/shield/detector.py, normalizer.py — Tests : test_regular.py — Barème : 4 pts (+1).

Cadre éthique

Alphabet abstrait $\{a, o, r\}$ (+ chiffres *leet*). Aucun contenu réel.

2.1 Contexte

Le premier réflexe d'un détecteur est de repérer un **motif** : « le mot contient le facteur **or** », un langage **régulier**. Avant de détecter, on **normalise** (défaire le *leetspeak* $4 \rightarrow a$, $0 \rightarrow o$, ...) : une relation **rationnelle** réalisée par un **transducteur fini**. On établit dès ce jour la première *limite due à la mémoire* (le renversement), qui annonce tout le projet.

2.2 Notions nouvelles

Objet formel — l'automate fini

Intuition. Une machine à états qui lit un mot lettre par lettre, sans autre mémoire que « où j'en suis » ; à la fin, elle répond *oui* ou *non*.

Définition formelle. Un AFD est $M = (Q, \Sigma, \delta, q_0, F)$ avec $\delta : Q \times \Sigma \rightarrow Q$; M accepte w si $\hat{\delta}(q_0, w) \in F$. Un langage est **régulier** s'il est reconnu par un tel automate (de façon équivalente : par un AFN, ou dénoté par une expression régulière).

Classe de problèmes. Les **problèmes de reconnaissance de mots** d'un langage régulier : décider l'appartenance $w \in L$. En pratique : recherche de motif, analyse lexicale, validation de formats simples, filtres.

Dans le projet. Décider « le mot contient le facteur **or** » (**SingularityDetector**) — un pur problème d'appartenance à un langage régulier.

Objet formel — le transducteur fini

Intuition. Un automate qui, en plus de lire, *écrit* : il **traduit** un mot en un autre au fil de la lecture.

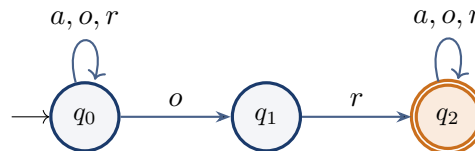
Définition formelle. Un transducteur séquentiel émet une sortie à chaque transition ; il réalise une **relation rationnelle** (ici une fonction).

Classe de problèmes. Les **transductions régulières** : normalisation, translittération, remplacement, nettoyage de surface.

Dans le projet. Normaliser le *leetspeak* ($4 \rightarrow a, 0 \rightarrow o, \dots$) avant la détection.

2.2.1 AFD et AFN

Un **AFD** lit un mot de gauche à droite ; dans chaque état, chaque lettre mène à *un seul* état. Un **AFN** autorise *plusieurs* transitions pour une même lettre : on accepte s'il *existe* un chemin acceptant. L'AFN « contient **or** » est très naturel : on « devine » où commence le facteur.



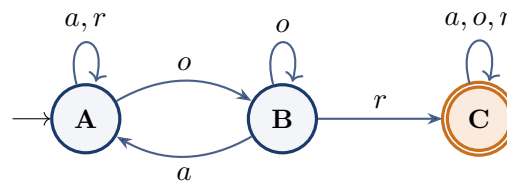
Non-déterminisme visible en q_0 : sur *o*, on peut *rester* en q_0 **ou** *aller* en q_1 .

2.2.2 Déterminisation par construction des sous-ensembles

Un état de l'AFD = un *ensemble* d'états de l'AFN. On part de $\{q_0\}$ et on ferme par transition :

état AFD	sur o	sur a, r	remarque
$S_0 = \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$	initial
$S_1 = \{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0\} / \{q_0, q_2\}$	o armé
$S_2 = \{q_0, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_2\}$	accepteur

Après renommage et minimisation, on retrouve **trois** états — l'AFD minimal :



Trace de l'AFD sur aor

pas	état	lit	transition
0	A	a	$A \xrightarrow{a} A$
1	A	o	$A \xrightarrow{o} B$
2	B	r	$B \xrightarrow{r} \mathbf{C}$ (accepté)

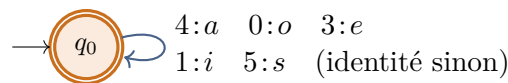
En B on est « armé » : un **r** immédiat bascule en C (absorbant) ; **aoa** retombe de B en A, rejeté.

2.2.3 Minimisation (Moore)

Deux états sont **indistinguishables** s'ils acceptent exactement les mêmes suffixes. On part de la partition finaux/non-finaux et on **raffine** tant que deux états d'un bloc mènent à des blocs différents. Le résultat (AFD minimal) est **unique** — lien direct avec Myhill–Nerode (jour 5).

2.2.4 Transducteur fini (FST) : normaliser le *leet*

Un AFD *reconnaît*; un **transducteur** *traduit* (chaque transition émet une sortie, notée $x : y$). Le normaliseur *leet* tient en **un seul état** (identité sur les lettres non listées).



Idempotence

$T(4or) = aor$, $T(r0le) = role$. La sortie ne contient plus de chiffre *leet*, donc $T(T(w)) = T(w)$: **idempotent**. La composition $T_2 \circ T_1$ se lit sur les paires d'états (**compose**).

2.2.5 Une limite déjà visible : one-way vs two-way

Le renversement $w \mapsto w^R$ est **impossible** en **one-way** : il faudrait mémoriser tout w avant d'émettre la première lettre — impossible à mémoire bornée. Une tête **two-way** le permet. C'est le premier exemple concret de *limite due à la mémoire* — thème qui culminera au jour 4.

2.3 A. Questions théoriques (à rédiger) — 2 pts

On pose $\Sigma = \{a, o, r\}$ et $L_1 = \{w \mid w \text{ contient le facteur } or\}$.

Q1.1 (0,5) — expression régulière.

Donner une regex dénotant *exactement* L_1 . *Livrable* : la regex + une phrase justifiant qu'elle capture tous les mots contenant **or** et aucun autre.

Q1.2 (1) — du non-déterminisme à l'AFD minimal.

Construire un AFN pour L_1 , le **déterminiser** (construction des sous-ensembles) puis le **minimiser** (raffinement de Moore). *Livrables* : (a) l'AFN dessiné; (b) la **table complète** des sous-ensembles atteints; (c) la **suite des partitions** de minimisation; (d) l'AFD minimal dessiné + son nombre d'états. *Indication* : vous devez retrouver **3 états**.

Q1.3 (0,5) — facteur vs sous-séquence.

Soit $L'_1 = \{w \mid w \text{ contient un } o \text{ suivi, pas forcément immédiatement, d'un } r\}$. Donner

sa regex, expliquer la différence avec L_1 , et prouver l'inclusion $L_1 \subseteq L'_1$. *Livrable* : regex + argument d'inclusion + un mot de $L'_1 \setminus L_1$.

2.4 B. Exercices de programmation — 2 pts

E1.1 (0,75) — détecteur AFD.

Dans `apps/shield/detector.py`, compléter la table δ de `_DFA_OR` (états A,B,C) et, dans `formlang/dfa.py`, `DFA.run/DFA.accepts`. *Contrainte* : `contains_or` passe par l'AFD, jamais par "or" in w. *Test* : `test_detector_or`.

E1.2 (0,5) — minimisation.

`DFA.minimize` (raffinement de Moore; utiliser `_completed()` fourni). *Test* : `test_minimisation_3_etats`.

E1.3 (0,5) — détermination.

`NFA.to_dfa` (sous-ensembles; `_eps_closure/_move` fournis). *Test* : `test_nfa_to_dfa_equivalence`.

E1.4 (0,25) — transducteur.

`leet_fst` (un état final, `identity_on_missing=True`) + `SequentialFST.transduce`.
Tests : `test_leet_idempotent`, `test_fst_composition`.

Pièges fréquents

- Oublier que `DFA.run` doit renvoyer `None` si une transition manque (mot rejeté), pas lever d'exception.
- Minimisation : ne pas compléter l'automate (états puits) avant de raffiner $\Rightarrow$ partitions fausses.
- FST : émettre le caractère *lu* plutôt que la *sortie* sur les transitions listées.

Sortie de référence

```
$ pytest -q tests/test_regular.py
7 passed
```

Item	Détail	Pts
Q1.1–Q1.3	regex, AFN $\rightarrow$ AFD $\rightarrow$ min, facteur vs sous-séquence	2
E1.1	détecteur AFD (<code>contains_or</code>)	0,75
E1.2	minimisation 3 états	0,5
E1.3	détermination	0,5
E1.4	FST leet + composition	0,25
Bonus	construction de Thompson (regex $\rightarrow$ AFN)	+1

Gate (règle non négociable)

`detector.py` instancie `formlang.dfa.DFA`; pas de "or" in w.

3 Jour 2 — Étage hors-contexte : grammaires & automate à pile

Modules : `formlang/pda.py`, `cfg.py` — App : `apps/shield/delimiters.py` — Tests : `test_cfg_nerode.py`, `test_regular.py` — Barème : 4 pts (+1).

Cadre éthique

$\Sigma_3 = \{a, o, r, [,], (,)\}$. Aucun contenu réel.

3.1 Contexte

Les **faux délimiteurs** imitent un bloc [...] ou un cadre (...), éventuellement imbriqués. Vérifier l'imbriication bien formée = reconnaître un langage de type **Dyck** : c'est **hors-contexte**, hors de portée d'un automate fini. Le nouvel ingrédient : la **pile**.

3.2 Notions nouvelles

Objet formel — la grammaire hors-contexte & l'automate à pile

Intuition. Une grammaire *engendre* des structures imbriquées par réécriture ; l'automate à pile est un automate fini **muni d'une pile** pour se souvenir de la profondeur d'imbriication.

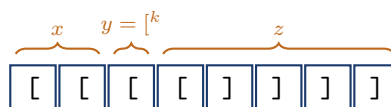
Définition formelle. Une CFG est $G = (N, \Sigma, R, S)$; un PDA ajoute une pile à un automate fini. Les deux décrivent la **même** classe : les langages **algébriques** (hors-contexte).

Classe de problèmes. La **reconnaissance et l'analyse syntaxique** de langages hors-contexte : équilibrage de parenthèses, expressions arithmétiques, structures récursives, *parsing* de langages de programmation.

Dans le projet. Vérifier l'imbriication bien formée des délimiteurs [], () — impossible pour un automate fini (jour 1), naturel pour une pile.

3.2.1 Le lemme de l'étoile (pompage)

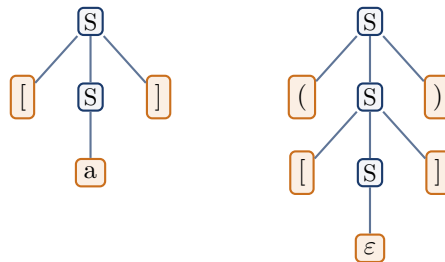
Outil pour **prouver qu'un langage n'est pas régulier**. Si L est régulier, il existe p tel que tout mot long $w = xyz$ ($|xy| \leq p$, $y \neq \varepsilon$) vérifie $xy^iz \in L$ pour tout i . Pour $D = \{[n]^n\}$, prendre $w = [p]^p$: comme $|xy| \leq p$, le bloc y n'est fait que de [, donc $xy^2z = [p+k]^p \notin D$.



Un automate fini a une mémoire **bornée** : il ne peut pas « compter » les ouvrantes.

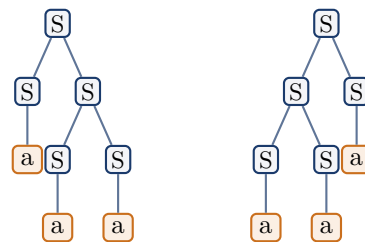
3.2.2 Grammaire hors-contexte et arbre de dérivation

Une CFG engendre par **réécriture** : $S \rightarrow SS \mid [S] \mid (S) \mid a \mid o \mid r \mid \varepsilon$. L'arbre de dérivation montre *comment* un mot est produit ($[a]$ et $([])$) :



3.2.3 Ambiguïté

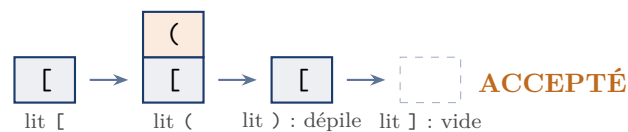
Une grammaire est **ambiguë** si un mot admet plusieurs arbres. $S \rightarrow SS$ n'est pas associée : **aaa** a (au moins) deux arbres.



On désambiguïse par $S \rightarrow TS \mid T$; $T \rightarrow [S] \mid (S) \mid a \mid o \mid r \mid \varepsilon$.

3.2.4 La pile : mémoire non bornée mais disciplinée

La **pile** (LIFO) est **non bornée** mais accessible seulement au **sommet**. Évolution sur $[a(r)]$ (on ignore **a**, **r**) : accepté car pile vide à la fin.



Un rejet : $([])$

On empile $($ puis $[$. Sur $)$, le sommet est $[\neq$ ouvrante attendue $(\Rightarrow$ **rejet**. L'imbriication « croisée » est mal formée : c'est la *pile* qui l'attrape, pas un compteur.

3.3 A. Questions théoriques (à rédiger) — 2,5 pts

Q2.1 (1,5) — non-régularité par pompage.

Montrer rigoureusement que $D = \{[n]^n \mid n \geq 0\}$ n'est pas régulier. *Livrable* : la preuve complète (choix de w , discussion de *toute* découpe xyz admissible, mot pompé hors de D), puis la conclusion : le `SingularityDetector` (AFD) ne peut pas vérifier l'imbrication.

Q2.2 (0,5) — grammaire.

Donner une grammaire G engendrant les prompts bien parenthésés sur Σ_3 (a, o, r libres entre délimiteurs). *Livrable* : les règles + un exemple de dérivation.

Q2.3 (0,5) — ambiguïté.

G est-elle ambiguë ? Si oui, donner **deux** arbres de dérivation de `aaa`, puis une grammaire **non ambiguë** équivalente.

3.4 B. Exercices de programmation — 1,5 pt

E2.1 (1) — automate à pile.

`DelimiterPDA.accepts avec une pile` (liste) : empiler sur ouvrant, dépiler sur fermant si le sommet correspond, ignorer a,o,r,e, accepter si pile vide. *Test* : `test_delimiters_pda`.

E2.2 (0,5) — génération bornée.

`CFG.generate(max_len)` : énumérer les mots *terminaux* de longueur $\leq \text{max_len}$. *Test* : `test_cfg_engendre_des_mots_equilibres`.

Piège majeur de E2.2

Avec $S \rightarrow SS \mid \dots \mid \varepsilon$, des formes purement non-terminales (S, SS, SSS, ...) ont **0 terminal** et ne sont jamais élaguées par la seule borne sur les terminaux $\Rightarrow$ **explosion mémoire**. Il faut borner **aussi** le nombre de non-terminaux (p.ex. $\leq 2 \cdot \text{max_len} + 2$) ; cette borne ne retire aucun mot de longueur $\leq \text{max_len}$.

Item	Détail	Pts
Q2.1	pompage $\{[n]^n\}$	1,5
Q2.2	grammaire bien parenthésée	0,5
Q2.3	ambiguïté + désambiguïsation	0,5
E2.1	PDA <code>well_parenthesized</code>	1
E2.2	génération bornée	0,5
Bonus	reconnaissance par CYK	+1

Gate (règle non négociable)

`delimiters.py` instancie `formlang.pda.DelimiterPDA`.

4 Jour 3 — Automates d'arbres (pivot) & hash-consing

Module : `formlang/tree.py` — Apps : `apps/morpho`, `apps/shield`, `apps/hashcons`
 — Tests : `test_tree.py`, `test_hashcons.py` — Barème : 5 pts (+2).

Cadre éthique

Volet Shield 100 % structurel : jetons abstraits, aucune attaque réelle.

4.1 Contexte

C'est le **pivot** du projet. Une séquence plate `p p r s` ne dit pas *comment* le mot est construit : la morphologie est **hiérarchique**, donc un **arbre**. On généralise l'automate fini : au lieu de suivre une *chaîne* d'états, on **fusionne** les états des enfants d'un nœud. Deux applications très différentes (morphologie, pare-feu structurel) *et* le hash-consinginstancient le **même** `formlang.tree`.

4.2 Notions nouvelles

Objet formel — l'automate d'arbres

Intuition. Un automate fini qui, au lieu de parcourir une *ligne*, « lit un **arbre** » des feuilles vers la racine, et décide si sa **forme** est admissible.

Définition formelle. Un BUTA est $A = (Q, \Sigma, Q_f, \Delta)$ avec des règles $f(q_1, \dots, q_n) \rightarrow q$; il accepte si la racine reçoit un état de Q_f . Il reconnaît les **langages d'arbres réguliers** (généralisation directe des langages réguliers de mots : un mot est un arbre unaire).

Classe de problèmes. La **validation de structures hiérarchiques** : conformité de schémas XML/JSON, vérification d'arbres de syntaxe abstraite (AST), typage, filtrage d'arbres (*tree matching*), *model checking* d'arbres.

Dans le projet. Valider la structure *morphologique* d'un mot (**morpho**) *et* décomposer/bloquer des structures dangereuses (**shield**) — deux applications d'un **même** automate d'arbres.

Objet formel — le hash-consing (partage de structure)

Intuition. Ne ranger chaque sous-arbre distinct **qu'une seule fois** et partager les copies : l'arbre devient un graphe orienté acyclique (**DAG**).

Définition formelle. Une fonction d'*internement* injective (terme $\mapsto$ identifiant) via une table canonique, garantissant un **partage maximal** et l'égalité structurelle en $O(1)$.

Classe de problèmes. La **représentation compacte** et la comparaison rapide de structures : dictionnaires morphologiques minimaux, diagrammes de décision (BDD), partage de sous-expressions en compilation.

Dans le projet. Mesurer le partage et le **taux de compression** des arbres morphologiques d'un vocabulaire.

4.2.1 Alphabet classé, termes, et automate d'arbres ascendant (BUTA)

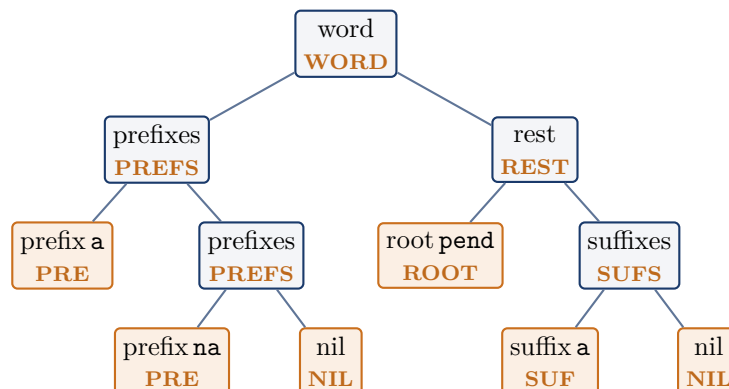
Chaque symbole a une **arité**. Un **BUTA** est $A = (Q, \Sigma, Q_f, \Delta)$ avec des règles $f(q_1, \dots, q_n) \rightarrow q$. On accepte si la **racine** reçoit un état de Q_f .

L'AFD généralisé

Un AFD lit un mot de gauche à droite : cas « tout en arité 1 » (chaîne). Le BUTA part des **feuilles** et **remonte** : à un nœud, il *combine* les états des enfants. • Code : `TreeAutomaton.run` (post-ordre ; REJECT si un enfant rejette ou si aucune règle).

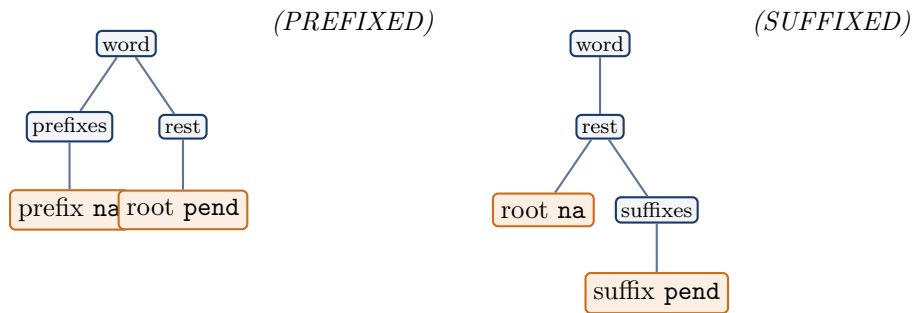
4.2.2 Exécution ascendante — exemple morphologique

Mot circonfixé **a-na-pend-a** : l'état de chaque nœud (**orange**) est calculé de bas en haut ; racine **WORD** ∈ $Q_f \Rightarrow$ accepté (classe CIRCUMFIXED).



4.2.3 Pourquoi un AFD de mots échoue : la frontière ne suffit pas

La **frontière** (yield) ne détermine pas la structure. Les deux arbres ci-dessous ont la **même** frontière **na pend** mais des classes différentes ; un AFD, qui ne lit que la frontière, est **aveugle** à la différence.

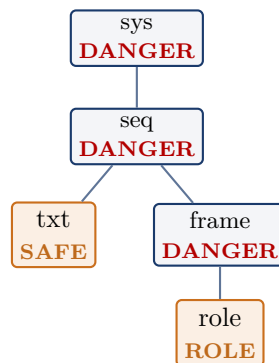


Théorème du yield — pont vers le jour 1

La frontière d'un mot morphologique accepté est exactement $p^* r s^*$, le langage **régulier** du jour 1 ; réciproquement, l'ensemble des frontières d'un automate d'arbres régulier est **hors-contexte**. Déterminisme $\Rightarrow$ exécution unique, coût $O(n)$.

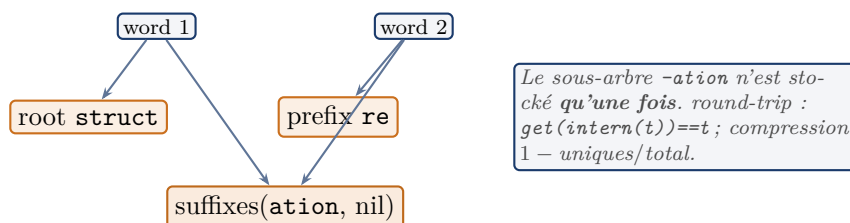
4.2.4 Le même moteur pour le Shield (100 % structurel)

`sys(seq(txt, frame(role)))` remonte à **DANGER** $\in Q_f \Rightarrow$ bloqué. Que de la forme.



4.2.5 Clôture par intersection (produit) et hash-consing (DAG)

Clôture par **intersection** : $A_1 \times A_2$ a pour états des *paires*, règles composante par composante. Le **hash-consing** donne à chaque sous-arbre distinct un **id unique** : sous-arbres identiques **partagés**, l'arbre devient un **DAG**.



4.3 A. Questions théoriques (à rédiger) — 2 pts

Q3.1 (0,5) — déterminisme & coût.

Montrer que `morpho_automaton` et `shield_automaton` sont déterministes (au plus une règle par (symbole, états enfants)). *Livrable* : l'argument + la conséquence (unicité de l'exécution, coût $O(n)$).

Q3.2 (0,5) — la frontière ne suffit pas.

Donner deux arbres de **même frontière** et de classes différentes, et expliquer pourquoi un AFD de mots ne peut les distinguer.

Q3.3 (0,5) — théorème du yield.

Montrer que la frontière d'un mot morphologique accepté est p^*rs^* , et faire le lien explicite avec le langage régulier du jour 1.

Q3.4 (0,5) — reduplication.

La reduplication (copie d'une partie de la racine) engendre-t-elle un langage d'arbres *régulier* ? Argumenter via $\{ww\}$ (non régulier). *Indice réel* : Culy 1985 — le bambara sort du hors-contexte.

4.4 B. Exercices de programmation — 3 pts

E3.1 (1) — moteur BUTA.

`TreeAutomaton.run` (post-ordre; REJECT si enfant rejeté ou règle absente) + accepts. *Débloque tout* `test_tree.py`.

E3.2 (0,5) — automate morphologique.

Règles Δ de `morpho_automaton` (feuilles prefix/root/suffix/nil; chaînes; rest, word). *Tests* : `test_morpho_accept_et_classes`, `test_morpho_rejet`.

E3.3 (0,5) — AttackDecomposer.

`_seq` (DANGER si l'un est DANGER ou si $\{OVR,ROLE\}$, sinon sévérité max) + `shield_automaton`. *Test* : `test_shield_blocage`.

E3.4 (0,5) — produit.

`product(A1,A2)` : $L = L(A_1) \cap L(A_2)$ (états = paires, règles composante par composante). *Test* : `test_produit_intersection`.

E3.5 (0,5) — hash-consing.

`CompactStore.intern` (partage) + `get` (round-trip). *Tests* : `test_round_trip_exact`, `test_sous_arbres_identiques_partages`, `test_compression_positive`.

Q3.5 (rapport) : $\text{compression} = 1 - \text{uniques}/\text{total}$; discuter pour quelle famille de langues (anglais isolant vs turc/finnois agglutinants) le partage est le plus élevé. Reportez vos **vrais** chiffres.

E3.6 (bonus) — P4.5 : « dangereux et doublement encodé ». Construire `enc_automaton` (compte les feuilles `enc`, plafonné à 2, final = $\{2\}$, même signature), puis `dangerous_and_double_encoded = product(shield_automaton(), enc_automaton())`. On bloque *ssi* l'arbre est dangereux et porte ≥ 2 encodages : illustration directe de la **clôture par intersection**. *Test* : `test_shield_double_encodage_P45`.

Pièges fréquents

- **run** : oublier de propager **REJECT** dès qu'un enfant rejette.
- **shield** : oublier une paire (x, y) dans les règles **seq** (le produit cartésien des états).
- **intern** : clé canonique incomplète (oublier **label**) $\Rightarrow$ faux partage et round-trip cassé.

Item	Détail	Pts
Q3.1–Q3.4	déterminisme, yield, reduplication	2
E3.1	run/accepts du BUTA	1
E3.2	automate morphologique + classify	0,5
E3.3	AttackDecomposer	0,5
E3.4	produit (intersection)	0,5
E3.5	hash-consing (intern + round-trip)	0,5
Bonus	infixation ; minimisation BUTA ; P4.5 (produit $A \times A_{enc}$)	+3

Gate (règle non négociable)

Les apps **instancient** `formlang.tree.TreeAutomaton/Term`. Toute réimplémentation perd les points d'intégration.

5 Jour 4 — Calculabilité : machine de Turing & machine universelle

Modules : formlang/turing.py, utm.py — App : apps/mtu — Tests : test_turing.py, test_calc.py — Barème : 4 pts (+1).

5.1 Contexte

On lève la dernière limite : une mémoire **libre**, lisible *et* ré-inscriptible n'importe où — la machine de Turing. Puis l'idée vertigineuse : **une seule** machine peut simuler **toutes** les autres si on lui donne leur description. C'est la machine **universelle**, ancêtre du processeur et de l'interpréteur.

5.2 Notions nouvelles

Objet formel — la machine de Turing

Intuition. Le modèle **ultime** : un ruban infini que l'on peut relire et réécrire librement. Il capture « tout ce qui est mécaniquement calculable ».

Définition formelle. $M = (Q, \Sigma, \Gamma, \delta, q_0, \sqcup, F)$ avec $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}$. Elle reconnaît les langages **récurivement énumérables** et calcule les fonctions **calculables**.

Classe de problèmes. **Tous** les problèmes de calcul : décision (avec la frontière *décidable/indécidable*) et calcul de fonctions. C'est la limite du calculable (thèse de Church–Turing).

Dans le projet. Réaliser l'arithmétique unaire (ADD, SUB) comme de **vraies** tables de transition, exécutées pas à pas.

Objet formel — la machine universelle

Intuition. Une machine de Turing qui **lit la description d'une autre** machine et l'exécute : un **interpréteur**.

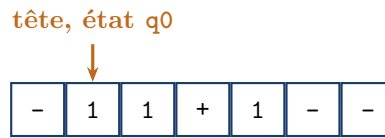
Définition formelle. $U(\langle M \rangle \#\# w)$ se comporte comme $M(w)$ (principe du *programme enregistré*).

Classe de problèmes. La **simulation universelle** — fondement de l'ordinateur programmable — et la **frontière de l'indécidabilité** (le problème de l'arrêt).

Dans le projet. Exécuter la calculatrice **par** U (`addition_via_utm`) : programme-donnée, pas de boucle de ruban réécrite.

5.2.1 La machine de Turing

$M = (Q, \Sigma, \Gamma, \delta, q_0, \sqcup, F)$ avec $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}$. À chaque pas : « dans l'état q , je lis $a \rightarrow$ j'écris b , je bouge de d , je passe en q' ». Différence avec la pile : ici on peut **relire et réécrire** le passé.



5.2.2 Une opération est une machine : trace de ADD sur 11+1

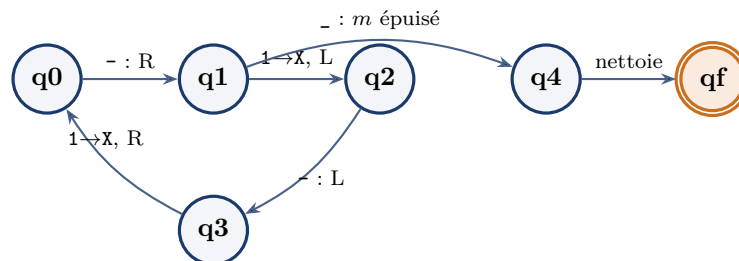
2+1 = 3, pas à pas

pas	état	ruban	action
0	q0	[1]1+1	lit 1 $\rightarrow$ R
1	q0	1[1]1+1	lit 1 $\rightarrow$ R
2	q0	11[+]1	le + devient 1, R, passe q1
3	q1	111[1]	lit 1 $\rightarrow$ R
4	q1	1111[_]	blanc : demi-tour, q2
5	q2	111[1]_	efface le dernier 1, qf
6	qf	111[_]	accepté ; 111 = 3

$+ \rightarrow 1$ *fusionne* les deux blocs, puis on retire **un** 1.

5.2.3 Lire une table par *phases* : la soustraction

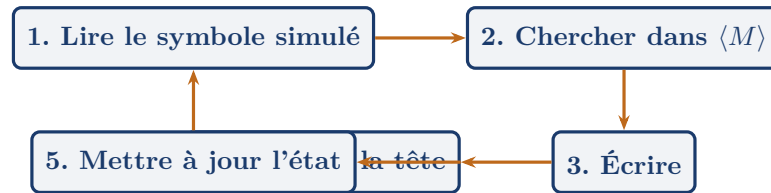
SUB apparie un 1 de m (marqué X) avec un 1 de n (barré, droite $\rightarrow$ gauche), jusqu'à épuiser m , puis nettoie.



Trois points délicats (à expliquer au rapport) : $(q_0, X) \rightarrow R$ et $(q_1, X) \rightarrow R$ (gérer ses propres marques); barrage **droite $\rightarrow$ gauche** (sortie contiguë); fin de m par un **blanc** (le ruban compte, pas un compteur).

5.2.4 Codage $\langle M \rangle$ et machine universelle

Une MT n'est qu'une **table finie** : on l'écrit comme une **chaîne** $\langle M \rangle$ (**injective**, **décodable**). U prend $\langle M \rangle \#\# w$ et **simule** M (programme enregistré). Un cycle :



Church–Turing, composition, limite

Thèse de Church–Turing : pas un théorème, étayée par l'équivalence de modèles (λ -calcul, fonctions récursives, RAM). On obtient $\times, \div$ par **composition** de MT (ce que fait U). *Revers* : HALT est **indécidable** (Turing 1936) ; par réduction, l'arrêt de U l'est aussi. *Universel $\neq$ omniscient*.

5.3 A. Questions théoriques (à rédiger) — 2 pts

Q4.1 (0,5) — encodage.

Détailler $\langle M \rangle$ (numéroter états/symboles, linéariser δ en quintuplets). *Livrable* : le schéma + preuve d'injectivité et de décodabilité.

Q4.2 (0,5) — la machine universelle.

Décrire U , l'invariant maintenu sur le ruban, et un cycle complet. *Livrable* : l'invariant + l'argument de correction (récurrence sur le nombre de pas).

Q4.3 (0,5) — surcoût (sourcé).

Multi-ruban $O(t \cdot |\langle M \rangle|)$; multi $\rightarrow$ mono $\leq$ quadratique ; citer Hennie–Stearns (ralentissement logarithmique). *Livrable* : les bornes **avec référence** (aucune constante « de mémoire »).

Q4.4 (0,5) — indécidabilité.

Réduction depuis HALT montrant que « U s'arrête sur $\langle M \rangle \#\# w$ » est indécidable ; commenter « universel $\neq$ omniscient ».

5.4 B. Exercices de programmation — 2 pts

E4.1 (0,5) — MT générique.

TuringMachine.run (ruban dict, arrêt sur état final / transition absente, option trace). *Test* : test_trace_disponible.

E4.2 (0,5) — machine universelle.

encode/decode (réciproques exactes) + UniversalTM.run. *Tests* : test_round_trip_encodage, test_utm_simule_comme_la_machine.

E4.3 (0,75) — calculatrice.

Tables ADD/SUB + Calculatrice (+, - par les MT ; *, / par composition ; chainer).

Tests : test_add_sub_machines_directes, test_calculatrice_exhaustif, test_chainage, test_division_par_zero.

E4.4 (0,25) — interpréteur universel.

UniversalInterpreter.run(M, w) qui encode M puis **délègue** à UniversalTM; addition_via_utm. *Test* : test_interpreteur_universel.

Pièges fréquents

- **run** : ne pas garantir l'arrêt (prévoir **max_steps** et lever en cas de boucle).
- **SUB** : oublier $(q_0, X) \rightarrow R / (q_1, X) \rightarrow R \Rightarrow$ blocage au 2^e tour.
- **encode/decode** : sortie non triée $\Rightarrow$ round-trip non déterministe.

Item	Détail	Pts
Q4.1–Q4.4	encodage, cycle U , surcoût sourcé , indécidabilité	2
E4.1	MT générique	0,5
E4.2	encode/decode + UniversalTM	0,5
E4.3	calculatrice (tests exhaustifs)	0,75
E4.4	interpréteur universel	0,25
Bonus	table monolithique de $\times$ validée par le simulateur	+1

Gate (règle non négociable)

apps/mtu ne réécrit aucune boucle de MT : il décrit des **tables** et les fait tourner via `formlang.turing/utm`. Surcoût **référéncé**.

6 Jour 5 — Intégration & Myhill–Nerode

Modules : `pipeline.py`, `formlang/myhill_nerode.py` — Tests : `test_pipeline.py`, `test_cfg_nerode.py` — Barème : 3 pts (+1).

6.1 Contexte

Deux visages : **assembler** (les étages coopèrent dans un **pipeline**) et **boucler la boucle théorique** — revenir sur « combien d'états faut-il vraiment ? » avec la congruence de Myhill–Nerode.

6.2 Notions nouvelles

Objet formel — la congruence de Myhill–Nerode

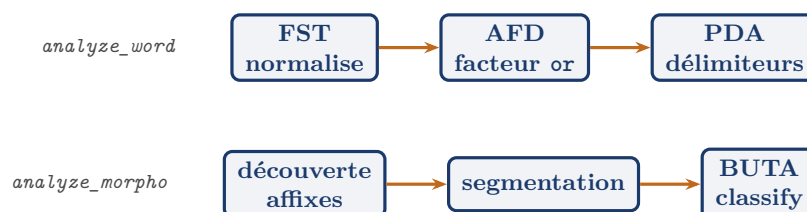
Intuition. Regrouper les mots qui « ont le même **avenir** » vis-à-vis du langage : une fois lus, ils mènent aux mêmes acceptations futures.

Définition formelle. $u \approx_L v$ ssi, pour tout suffixe w , $uw \in L \Leftrightarrow vw \in L$. C'est une équivalence **invariante à droite**, et L est régulier ssi son **indice** (nombre de classes) est fini.

Classe de problèmes. La **caractérisation** des langages réguliers et la **mini-misation** : l'indice égale le nombre d'états de l'AFD minimal ; identifier les états équivalents, c'est identifier les classes.

Dans le projet. Retrouver les **3 classes** de L_1 (= 3 états de l'AFD minimal du jour 1) et refermer ainsi le fil rouge théorique.

6.2.1 Intégration : deux chaînes, un même cœur



6.2.2 La congruence de Myhill–Nerode

$u \approx_L v$ si, **pour tout** suffixe w , $uw \in L \Leftrightarrow vw \in L$: l'*avenir* après u ou v est identique. Équivalence **invariante à droite**.

Le théorème — le fil rouge se referme

L est régulier **ssi** $\approx_L$ a un nombre **fini** de classes, et ce nombre (l'**indice**) est **exactement** le nombre d'états de l'AFD **minimal**. Minimiser (jour 1), c'était calculer ces classes.

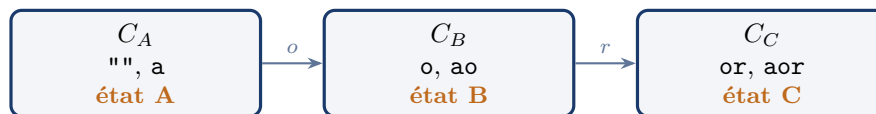
6.2.3 En pratique : signatures sur suffixes témoins

Avec $S = \{\varepsilon, r, or\}$, la **signature** d'un mot est $(\mathbf{1}[ws \in L_1])_{s \in S}$:

Trois signatures = trois classes

mot w	$w\varepsilon$	wr	wor	classe
""	0	0	1	A
a	0	0	1	A
o	0	1	1	B
ao	0	1	1	B
or	1	1	1	C
aor	1	1	1	C

Signatures $(0, 0, 1)$, $(0, 1, 1)$, $(1, 1, 1)$: exactement **3 classes** $\Rightarrow$ 3 états.



6.3 A. Questions théoriques (à rédiger) — 1 pt

Q5.1 (0,5)

Énoncer Myhill–Nerode et relier le nombre de classes de L_1 au nombre d'états de l'AFD minimal du jour 1. *Livrable* : énoncé + le lien chiffré (3).

Q5.2 (0,5)

Expliquer le critère « indistinguishables par tous les suffixes témoins » et son lien avec la **fusion d'états** à la minimisation.

6.4 B. Exercices de programmation — 2 pts

E5.1 (1) — pipeline.

`analyze_word (FST→AFD→PDA)` et `analyze_morpho (discover→segment_to_tree→classify)`.
Tests : `test_pipeline_word`, `test_pipeline_morpho`.

E5.3 (1) — classes de Nerode.

`nerode_classes(accepts, words, suffixes)` : regrouper par signature `tuple(accepts(w+s) for s in suffixes)`. *Test* : `test_nerode_trois_classes`.

Sortie de référence

```
$ pytest -q
28 passed
$ python pipeline.py -word 4or
  normalisé(FST) : aor  facteur_or(AFD) : True  délimiteurs_ok(PDA)
: True
```

Item	Détail	Pts
Q5.1–Q5.2	Myhill–Nerode + lien minimisation	1
E5.1	pipeline (mot + morpho)	1
E5.3	classes de Nerode	1
Bonus	mesure hash-consing sur 10 000 mots	+1

Gate (règle non négociable)

Le pipeline **appelle les apps** (quiinstancient le cœur) ; il ne reprogramme aucun automate.

7 Bibliographie (vérifiée)

Les références marquées (**libre**) sont disponibles gratuitement.

Automates finis & langages. M. Sipser, *Introduction to the Theory of Computation*, 3^e éd., Cengage, 2012. • J. Hopcroft, R. Motwani, J. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3^e éd., Pearson, 2006.

Automates d'arbres. H. Comon *et al.*, *Tree Automata Techniques and Applications (TATA)*, 2007 (**libre : tata.gforge.inria.fr**). • F. Gécseg, M. Steinby, *Tree Automata*, 1984 ; rééd. (**libre : arXiv :1509.06233, 2015**). • J. W. Thatcher, J. B. Wright, *Math. Systems Theory* 2(1) :57–81, 1968 (*fondateur*). • J. Doner, *JCSS* 4(5) :406–451, 1970.

Applications des arbres. M. Murata *et al.*, *ACM TOIT* 5(4), 2005 (schémas XML). • A. V. Aho, M. Ganapathi, S. Tjiang, *ACM TOPLAS* 11(4), 1989 (filtrage d'arbres). • A. Bouajjani *et al.*, *ENTCS* 149(1), 2006.

Morphologie computationnelle. K. Koskenniemi, *Two-Level Morphology*, Helsinki, 1983. • K. R. Beesley, L. Karttunen, *Finite State Morphology*, CSLI, 2003. • Z. S. Harris, *Language* 31(2) :190–222, 1955. • J. Goldsmith, *Computational Linguistics* 27(2) :153–198, 2001. • E. Roche, Y. Schabes (dir.), *Finite-State Language Processing*, MIT Press, 1997.

Réduplication & limites. C. Culy, « The complexity of the vocabulary of Bambara », *Linguistics and Philosophy* 8 :345–351, 1985. • B. Roark, R. Sproat, *Computational Approaches to Morphology and Syntax*, Oxford, 2007.

Hash-consing. J.-C. Filliâtre, S. Conchon, « Type-safe modular hash-consing », *ML Workshop*, 2006. • J. Daciuk *et al.*, *Computational Linguistics* 26(1), 2000.

Calculabilité. A. M. Turing, « On Computable Numbers... », *Proc. London Math. Soc.*, s2-42 :230–265, 1936-37. • F. C. Hennie, R. E. Stearns, « Two-Tape Simulation of Multitape Turing Machines », *J. ACM* 13(4) :533–546, 1966. • S. Arora, B. Barak, *Computational Complexity : A Modern Approach*, Cambridge, 2009.